Charter

A programming language for software that an AI writes and a machine checks

Overview of what it is, how it works and why we would use it. State of the project on November 9, 2026. Source: github.com/adaPlu/Charter.

> **In short.** A person says in plain words what the software should do. An AI writes that down as a small, strict program in the Charter language. Ordinary, predictable tools then check the program, prove what they can about it, and run it. No AI is involved once the software is running.
>
> The bet is that an AI makes fewer mistakes in a language built this way, and that the mistakes it does make are caught before the software runs. The tools to check and run Charter programs are built and working. The experiment that tests the bet is ready but has not been run.

1. What Charter is

A Charter program is a charter in the ordinary sense of the word: a written statement of the rules. It says five things about a piece of software.

The program states	Charter word	Example from a to-do app
What data is stored	`entity`	A task has an owner, a title, an optional due date and a done flag.
Who may do what	`permission`	A person owns a task if they are its owner.
Which actions exist, and what each one changes	`rule`	Completing a task is allowed only for its owner, only if it is not already done, and sets the done flag.
What must always be true	`invariant`	A store-credit balance is never negative.
What the software is for, in plain words, and whether a person agreed to it	`requirement`	"Each person sees and changes only their own tasks." Confirmed.

Charter is aimed at software that is mostly data, rules and permissions: business applications, web services, workflows and game servers. In that kind of software, most bugs are broken rules, not broken algorithms. It is the wrong tool for heavy computation, device drivers or game engines.

Very little of Charter is invented. Its ideas come from existing, well-studied systems: transactions from databases, guarded rules from formal modelling languages, automatic proofs from verification tools, and deny-by-default permissions from policy languages. What is new is the combination, and three additions: requirements live inside the program, the grammar is small enough that an AI's output can be held to it, and AI edits are meant to be small, typed changes instead of free rewrites of text.

2. How it works

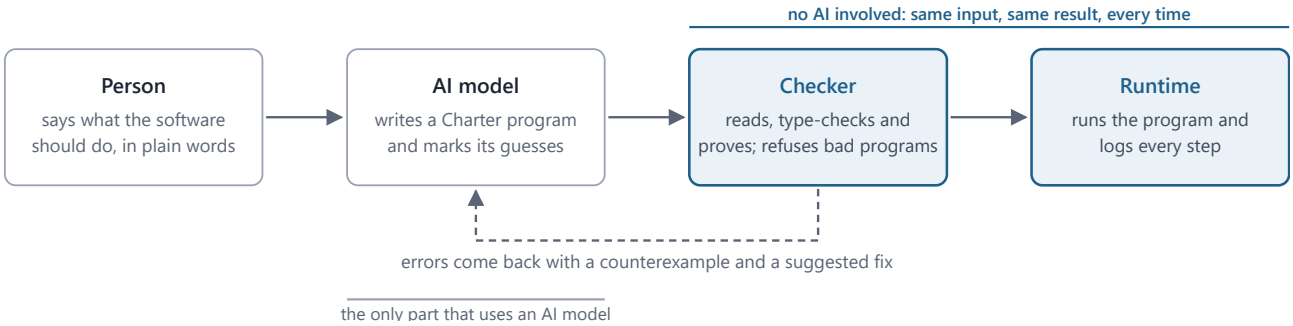


Figure 1. The path from a request to running software. The checker and runtime are built. The command that connects a person's words to the AI (`charter ask`) is planned; today the AI step is driven by the test harness.

2.1 A complete program

This is a whole Charter program, taken unchanged from the project (`bench/tasks/p12-store-credit/reference.charter`). It keeps a store-credit balance for each person.

```
module credit@0.1.0
principal Person

entity Person { name: Text, balance: Int where >= 0 = 0 }
enum CreditError { InsufficientCredit }

rule Register(name: Text where len(trim(it)) >= 1): Id<Person>
  permit system
  create Person { name = name } as p
  return p.id

rule Grant(customer: Person, cents: Int where it >= 1 and it <= 100000): Int
  permit system
  set customer.balance = customer.balance + cents
  return customer.balance

rule Spend(cents: Int where >= 1): Int rejects CreditError
  permit anyone
  let me = caller
  guard me.balance >= cents else reject InsufficientCredit
  set me.balance = me.balance - cents
  return me.balance

query Balance(): Int permit anyone = caller.balance

invariant NeverNegative: Person.balance >= 0
```

Line	What it means
`principal Person`	The people who sign in are `Person` records. Inside a rule, `caller` is the person making the request.
`balance: Int where >= 0 = 0`	A balance is a whole number that is never below zero, and starts at zero. The limit is part of the type, so it applies everywhere.
`permit system`	Only the operator may register people or grant credit. Anything not permitted is denied.
`cents: Int where it >= 1 and it <= 100000`	A grant must be between 1 and 100,000 cents. A request outside that range is rejected before the rule runs.
`guard … else reject InsufficientCredit`	Spending more than the balance is refused with a named reason, and nothing changes.
`query Balance()`	A query reads data and can never change it.
`invariant NeverNegative`	No action, present or future, may leave any balance below zero. This is checked after every change.

2.2 What happens when a rule is called

Rules are the only way to change stored data. Every call to a rule is one *step*, and every step goes through the same sequence. If any check fails, the step is rejected with a named reason and the data is left exactly as it was.

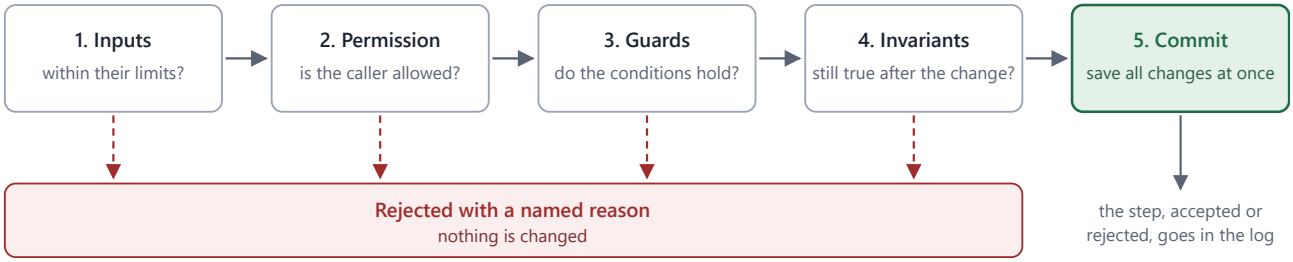


Figure 2. One step. A change is saved completely or not at all.

Because every step is recorded, the whole history can be replayed on an empty database, and it must reproduce every outcome and the exact final data. That is how the project checks that running a Charter program is fully predictable.

2.3 The checker refuses programs it cannot trust

Before a program runs, `charter check` does three things: it reads the program against the grammar, checks every name and type, and then tries to *prove* that certain errors cannot happen. For the store-credit program above it reports:

```
✓ parsed (1 module, 8 declarations)
✓ names and types
✓ proof obligations: 1 proven, 1 kept as runtime checks
    proven         me.balance - cents stays within the Int range  (line 21)
    runtime check  customer.balance + cents stays within the Int range  (line 14)
```

When it cannot prove something that matters, it refuses the program and shows an input that would break it. Here a calculator divides without first checking the divisor:

```
error CH0401  cannot prove that b is not zero
    known: nothing
    counterexample: b = 0
    fix (add_guard): check the divisor first, for example: if b == 0 then … else …
```

The message is written for an AI as much as for a person. It names the problem, gives a concrete failing case and suggests the kind of fix, so a model can repair its own program without guessing.

2.4 Requirements are part of the program

Each requirement is a line in the program with a status. `confirmed` means a person asked for it. `inferred` means the AI filled a gap with a guess. Code points back to the requirement it serves with `req R-5`, and the checker reports every guess that nobody has approved:

```
requirement R-5 confirmed "Each person sees and changes only their own tasks."
requirement R-7 inferred  "Done tasks cannot be reopened."

⚠ 4 requirement(s) not confirmed: R-6 (inferred), R-7 (inferred), R-8 (inferred), R-10 (inferred).
  A release build is blocked until a person confirms them.
```

This is the part of Charter aimed at the hardest problem with AI-written software: the AI quietly deciding something you never asked for.

3. Why we would use it

Each part of the design answers a specific way that AI-written software goes wrong today.

Problem today	What Charter does about it	State
The AI writes code that does not compile, or is subtly wrong.	The language is small (83 grammar rules) and regular, its words are borrowed from TypeScript and SQL, and every program has exactly one correct layout. There are fewer ways to go wrong.	Built, unproven

A check is forgotten in one of the many places that change data.	Data changes only through rules. Every rule call passes permission, guards and invariants, and an invariant is written once and applies to every rule.	Built
Bugs are found only when the software is running.	The checker proves that some errors cannot happen, and refuses the program with a counterexample when it cannot.	Partly built
The AI guesses what you meant and you never find out.	Guesses are marked `inferred`, listed by the checker, and are meant to block a release until a person confirms them.	Partly built
An AI in a running system is unpredictable and hard to audit.	No AI runs in production. The same request on the same data always gives the same result, and every step is logged and can be replayed.	Built
A small change request breaks something unrelated.	AI edits are meant to be small, typed changes to one part of the program, each with a report of what else is affected.	Planned

"Partly built" in detail: division by zero and type limits are proven now, while invariants are checked after every change and will be proven ahead of time in milestone 3. Guesses are marked and reported now, but the release block is only a warning, and nothing yet stops an AI from marking its own guess as confirmed.

Why it should work, and how we will know

The design rests on four claims. Each has a measurement and a bar it must clear against TypeScript, with the same AI models, the same tasks and the same budget.

	Claim	Bar to clear
H1	AI models write valid programs more often in a small, strict language.	Valid on the first try at least 15 points more often than TypeScript, and passing hidden tests at least as often.
H2	Structured edits cause fewer accidental changes than editing text.	At least 50% fewer unrelated behaviours changed per request.
H3	Writing rules, permissions and invariants directly catches more mistakes before running.	At least twice as many seeded mistakes caught as strict TypeScript with a linter.
H4	Reading the program back in plain words lets people spot a misunderstanding faster.	More misreadings spotted, in less time, than by reading generated TypeScript.

None of these has been measured yet. The honest position is that Charter is a well-specified bet with working tools, not a proven result. The design says plainly that Charter fails as an AI-native language if H1 and H2 both fail, even if H3 passes.

4. Where the project stands

Working now

- The grammar, frozen, with 27 recorded amendments
- Parser and formatter
- Name and type checker
- Automatic proofs for division by zero and type limits
- Stored data, with each rule call saved completely or not at all
- Permissions, guards and invariants checked on every step
- Tests and randomised property tests inside the program
- Any program served as a web service (`charter serve`)
- Step log and replay

Not built yet

- Proving invariants ahead of time (milestone 3)
- Outside effects such as payments and email: they are queued but not sent (milestone 4)
- The plain-language interface, `charter ask` (milestone 7)
- A release build that refuses unconfirmed requirements
- Structured AI edits with impact reports
- Sign-in: the web service trusts a development-only header

The project checks itself on every change, on Linux and Windows: 21 reference programs with their expected results, and 102 tests that include about 1,900 randomly generated programs per run.

The first experiment (the H1 pilot)

The pilot is a small first test of claim H1, and it is allowed to stop the project. A model is given the same task in both languages. Its program is checked with each language's real tool, given up to three chances to repair itself from the error messages, then run against hidden tests over the web. If Charter's first-try gain is under 5 points *and* its programs pass fewer hidden tests, development stops for a design review.

Piece	State
Test harness	Built and tested, including a trial run with a small local model.
Tasks	22 tasks written and frozen, each with 4 hidden test scenarios. The project owner reviewed and approved all 22 on November 9, 2026. The design asks for a reviewer independent of the project, so the pilot report must say this was not one.
Hand-written reference answers	TypeScript passes all 22 tasks. Charter passes 19 of 22 completely; the three gaps are recorded as limits of the language, not hidden.
Still needed	Access to the AI models, and a budget estimated at $15 to $40. An independent review of the tasks is still worth adding.

What Charter gives up

These are deliberate trade-offs, plus limits that writing the reference answers exposed.

- **General-purpose algorithms.** Loops are bounded and heavy computation must be handed to outside code.
- **Text handling is weak.** Reading inside a piece of text takes an awkward recursive function. This is one cause of the three reference gaps.
- **Low-level control.** No threads, no memory management.
- **Common words are reserved.** Names such as `to`, `by`, `set`, `order` and `result` cannot be used for your own data.
- **It is new.** AI models have seen enormous amounts of TypeScript and no Charter. The familiar-looking syntax is meant to offset this, and the pilot measures whether it does.

---

5. Try it

You need Node 22.18 or later. Run these from the top folder of the repository.

Install once

```
npx pnpm@9.15.4 install
```

Check a program, then run the tests written inside it

```
node packages/cli/src/main.ts check examples/todo/todo.charter
node packages/cli/src/main.ts test  examples/todo/todo.charter
```

See the checker refuse a program, with a counterexample

```
node packages/cli/src/main.ts check conformance/cases/n005-division-unproven/input.charter
```

Run the to-do app as a web service

```
node packages/cli/src/main.ts serve examples/todo/todo.charter --db todo.db --port 8080
```

The full design, all nine phases, is in `docs/charter-design.md`. Findings that contradicted the design are recorded in `docs/design/10-errata.md`.